

PROJETO DE SOFTWARE LIVRE
REPOSITÓRIO `WILLIANDLIMA/GPS_HTML`

Edição oficial

GPS Tracker

*Relatório Técnico de um Aplicativo de Geolocalização, Medição
de Distância Percorrida e Bússola de Norte Verdadeiro*

Aplicação híbrida Web / PWA / Android (Capacitor)
Documentação de arquitetura, algoritmos e validação
Relativo à versão 1.4.1 do software · Emitido em julho de 2026

SÃO PAULO — 2026

RESUMO

Este relatório descreve, em detalhe técnico, o desenvolvimento do **GPS Tracker**, um aplicativo de geolocalização construído a partir de um único documento `index.html` autocontido e progressivamente enriquecido até tornar-se, simultaneamente, uma página web, um *Progressive Web App* (PWA) instalável e um aplicativo Android nativo empacotado com Capacitor. O sistema cumpre três funções centrais: (i) obtenção e refinamento da posição corrente por meio da *Geolocation API*; (ii) cálculo da distância percorrida durante um trajeto, com um método híbrido que combina integração da velocidade Doppler, rejeição de discontinuidades (*spikes*) e um limiar de movimento proporcional à incerteza; e (iii) uma bússola de norte verdadeiro que corrige o rumo magnético do sensor de orientação pela declinação magnética local, calculada de forma offline com o modelo World Magnetic Model 2025 (WMM2025). O documento apresenta a fundamentação teórica, a arquitetura, a descrição comentada do código, a metodologia de validação por simulação numérica e os resultados obtidos, além das limitações e dos trabalhos futuros. Demonstra-se que o método híbrido de distância reduz a superestimação típica da soma de posições ruidosas de fatores de 2 a 11 vezes para erros da ordem de poucos pontos percentuais, e que a declinação calculada coincide, dentro de meio grau, com a referência oficial do NOAA.

PALAVRAS-CHAVE: Geolocalização. GPS. Distância percorrida. Fórmula de Haversine. Filtragem de sinal. Declinação magnética. WMM2025. Capacitor. PWA.

SUMÁRIO

- 1 Introdução
- 2 Fundamentação teórica
- 3 Materiais e métodos
- 4 Arquitetura do sistema

5	Desenvolvimento e descrição do código
6	Validação e resultados
7	Limitações
8	Trabalhos futuros
9	Conclusão
—	Referências

1 INTRODUÇÃO

1.1 Contextualização

A popularização dos receptores de sinais de satélite em telefones celulares tornou a geolocalização um recurso onipresente. Aplicativos de corrida, ciclismo, caminhada e logística dependem da capacidade de determinar a posição do dispositivo e, a partir de uma sequência dessas posições, estimar a distância percorrida. Contudo, o sinal de posicionamento é intrinsecamente ruidoso: mesmo parado, o ponto reportado oscila alguns metros, e essa oscilação, quando somada ingenuamente ao longo de um trajeto, infla a distância medida. O presente projeto nasceu de uma demanda simples — *obter a posição atual e calcular a distância percorrida a partir dela até que o operador encerre* — e evoluiu, de forma incremental e orientada a testes reais em campo, até um sistema multiplataforma com tratamento estatístico do sinal e recursos auxiliares de orientação.

1.2 Problema

Três problemas técnicos guiaram o desenvolvimento. O primeiro é a **qualidade da posição inicial**: navegadores frequentemente entregam, no primeiro instante, uma estimativa grosseira baseada em endereço IP ou redes Wi-Fi, com erro na casa de quilômetros. O segundo, e mais crítico, é a **superestimação da distância** causada pelo acúmulo de ruído de posição. O terceiro é a **continuidade do rastreamento** quando o

aparelho é bloqueado ou o aplicativo vai a segundo plano — situação em que um aplicativo puramente web é suspenso pelo sistema operacional.

1.3 Justificativa

Ferramentas consagradas, como o Strava, aplicam suavização leve e são publicamente conhecidas por superestimar a distância em torno de 2% a 3%. Há, portanto, espaço para um tratamento mais rigoroso, aproveitando informações que a maioria das somas ingênuas ignora — em especial a **velocidade instantânea derivada do efeito Doppler**, cuja incerteza é muito menor que a da diferença de posições. Construir esse tratamento em um único arquivo, sem dependências de execução externas, favorece a auditabilidade e a portabilidade entre web e aplicativo nativo.

1.4 Objetivos

Objetivo geral

Desenvolver um aplicativo multiplataforma capaz de obter a posição corrente, calcular com precisão a distância percorrida e indicar o norte geográfico, funcionando tanto no navegador quanto como aplicativo Android com rastreamento em segundo plano.

Objetivos específicos

- Refinar a posição corrente descartando estimativas grosseiras de rede;
- Projetar e validar um método de cálculo de distância robusto ao ruído do GPS;
- Prover continuidade do rastreamento com tela bloqueada por meio de empacotamento nativo;
- Implementar uma bússola de norte verdadeiro com correção da declinação magnética local;
- Automatizar a compilação e a distribuição do aplicativo Android;
- Validar quantitativamente cada decisão por simulação numérica e testes de execução.

1.5 Estrutura do relatório

A Seção 2 apresenta a fundamentação teórica. A Seção 3 descreve os materiais, o método de desenvolvimento incremental e a estratégia de validação. A Seção 4 detalha a arquitetura. A Seção 5 documenta o código, algoritmo a algoritmo. A Seção 6 reúne os resultados de validação. As Seções 7 e 8 discutem limitações e trabalhos futuros, e a Seção 9 conclui.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Posicionamento por satélite e a incerteza do sinal

Os sistemas globais de navegação por satélite (GNSS), dos quais o GPS é o mais conhecido, determinam a posição de um receptor por trilateração dos tempos de propagação de sinais de múltiplos satélites. Em um telefone, o resultado é fundido com sensores inerciais e com posicionamento por redes (Wi-Fi e torres de celular). A *Geolocation API* expõe, para cada leitura, um campo de **precisão** (`accuracy`), interpretado como o raio, em metros, de um círculo de confiança de 68%. Esse valor é a variável central para decidir quanto confiar em cada ponto: leituras com precisão de dezenas de metros contêm ruído comparável ao próprio deslocamento de um pedestre entre duas amostras.

2.2 Geolocation API e contexto seguro

A obtenção de posição no navegador exige um *contexto seguro* (HTTPS ou `localhost`). A API oferece `getCurrentPosition`, para uma leitura pontual, e `watchPosition`, para um fluxo contínuo. Cada objeto de posição contém latitude, longitude, precisão, e, quando disponível, altitude, rumo (*heading*) e velocidade (*speed*), esta última em metros por segundo. A disponibilidade da velocidade é decisiva para o método de distância adotado neste trabalho.

2.3 Distância geodésica: a fórmula de Haversine

A distância sobre a superfície aproximadamente esférica da Terra entre dois pontos de coordenadas geográficas é dada pela fórmula de Haversine, que calcula o comprimento

do arco de círculo máximo. Sendo φ a latitude e λ a longitude em radianos, e R o raio médio terrestre (6.371.000 m), a distância d é:

$$a = \sin^2(\Delta\varphi/2) + \cos \varphi_1 \cdot \cos \varphi_2 \cdot \sin^2(\Delta\lambda/2)$$
$$d = 2R \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a})$$

Para as distâncias curtas entre amostras consecutivas de GPS (metros a dezenas de metros), a aproximação esférica é mais que suficiente; o erro frente a modelos elipsoidais é desprezível nessa escala.

2.4 Velocidade Doppler e integração temporal

Além da posição, o receptor estima a velocidade a partir do **desvio Doppler** das portadoras dos satélites. Essa medida é obtida de forma independente da posição e costuma ser substancialmente mais precisa: enquanto a posição oscila metros quando o aparelho está imóvel, a velocidade reportada tende a zero. Disso decorre uma estratégia poderosa: estimar a distância como a **integral da velocidade no tempo**, $d = \int v \, dt$, aproximada de forma trapezoidal entre amostras. Essa formulação é praticamente imune ao tremor de posição e embute, naturalmente, a detecção de repouso (velocidade abaixo de um limiar).

2.5 Declinação magnética e o modelo WMM2025

A bússola de um telefone mede o norte *magnético* por meio de um magnetômetro. O norte *geográfico* (verdadeiro) difere do magnético por um ângulo chamado **declinação magnética**, que varia com a posição sobre o globo e lentamente no tempo. Na região de São Paulo, por exemplo, essa diferença é da ordem de -22° , magnitude que, ignorada, tornaria a bússola inútil. O padrão de fato para calcular a declinação é o *World Magnetic Model* (WMM), um modelo de harmônicos esféricos do campo geomagnético publicado com validade quinquenal. A edição vigente, WMM2025, cobre o período de 2025 a 2030. Neste projeto, o modelo é embarcado e avaliado offline.

2.6 PWA e a ponte web-nativo do Capacitor

Um *Progressive Web App* é uma página web que, por meio de um *manifest* e de metadados apropriados, pode ser instalada na tela inicial e aberta em tela cheia. Todavia, um PWA continua sujeito às políticas de suspensão do navegador. Para obter rastreamento real em segundo plano, recorre-se ao **Capacitor**, que empacota os mesmos arquivos web em um projeto nativo e expõe, à página, plugins que acessam capacidades do sistema operacional. Assim, um único `index.html` serve tanto ao navegador quanto ao aplicativo, detectando em tempo de execução o ambiente em que roda.

2.7 Rastreamento em segundo plano no Android

O Android permite execução contínua em segundo plano apenas por meio de um *serviço em primeiro plano*, associado a uma notificação persistente e a permissões específicas de localização (incluindo `ACCESS_BACKGROUND_LOCATION` e `FOREGROUND_SERVICE_LOCATION`). O plugin comunitário de *background geolocation* encapsula esse serviço, entregando à camada web um fluxo de posições mesmo com a tela bloqueada.

3 MATERIAIS E MÉTODOS

3.1 Ferramentas e tecnologias

Tabela 1 — Principais componentes tecnológicos do projeto.

Componente	Tecnologia / versão	Papel
Camada de aplicação	HTML5, CSS3, JavaScript (ES2020)	Interface e lógica, sem framework
Empacotamento nativo	Capacitor 8.4.2	Ponte web → Android
GPS em segundo plano	@capacitor-community/background-geolocation 1.2.26	Serviço em primeiro plano
Declinação magnética	geomagnetism (WMM2025), Apache-2.0	Norte verdadeiro (offline)
Build Android	Gradle 8.14.3, AGP 8.13, compileSdk 36, minSdk 24	Geração do APK
Integração contínua	GitHub Actions	Compilação e publicação do APK na nuvem
Hospedagem web	GitHub Pages	Versão web/PWA

3.2 Modelo de desenvolvimento e versionamento

O desenvolvimento seguiu um modelo **incremental e iterativo**, com cada funcionalidade ou correção validada em campo antes do próximo passo. Adotou-se **versionamento semântico** (MAJOR.MINOR.PATCH), registrado em um arquivo de *changelog* e refletido no número exibido no rodapé do aplicativo. A Tabela 2 resume a evolução das versões que impactaram diretamente o cálculo de distância — o núcleo funcional do sistema.

Tabela 2 — Evolução do método de cálculo de distância.

Versão	Mudança essencial no método
1.0.0	Soma de posições (Haversine) com limiar fixo de 0,5 m
1.0.1–1.0.5	Ajustes sucessivos do limiar e da precisão aceita
1.2.3	Limiar proporcional à precisão ($0,75 \times$ pior precisão)
1.3.3	Filtro por velocidade (auto-pausa) e limiar de $1,5 \times$ precisão
1.4.0	Método híbrido: integração de velocidade + rejeição de <i>spike</i>
1.4.1	Botão de zerar sessão e exibição de versão

3.3 Integração e entrega contínua

Como o ambiente de desenvolvimento não dispunha do SDK Android, adotou-se a compilação na nuvem. A cada *push* ao ramo principal, um fluxo de trabalho do GitHub Actions instala as dependências, sincroniza os arquivos web no projeto nativo, compila o APK de depuração com o Gradle e o publica como ativo de uma *release* de download direto. O rodapé do aplicativo passa a exibir, além do número de versão, o *hash* do *commit* e a data, injetados no momento da compilação, de modo que cada APK seja rastreável até a linha exata do histórico que o originou.

3.4 Estratégia de validação por simulação

Dada a impossibilidade de exercitar o sensor físico no ambiente de desenvolvimento, cada decisão algorítmica foi validada por **simulação numérica**. Trajetos sintéticos (retos, sinuosos e imóveis) foram amostrados e perturbados por **ruído correlacionado** — um processo autorregressivo que reproduz a natureza lentamente derivante do erro de GPS, mais realista que ruído independente. Compararam-se os métodos "somente posição", "somente velocidade" e "híbrido" contra a distância verdadeira conhecida do trajeto. Complementarmente, o código real do aplicativo foi exercitado em navegador de teste automatizado, injetando-se sequências de posições com velocidade e marcas de tempo realistas.

4 ARQUITETURA DO SISTEMA

4.1 Visão geral

O sistema é organizado em torno de um único documento autocontido. A lógica reside em uma função imediatamente invocada (IIFE) que encapsula o estado e expõe apenas os manipuladores de eventos da interface. Não há dependências de execução carregadas remotamente: o único artefato externo é o pacote do modelo geomagnético, embarcado como arquivo local. Essa escolha maximiza a portabilidade — os mesmos bytes servem à web, ao PWA e ao aplicativo nativo — e a auditabilidade.

4.2 Detecção de ambiente

Em tempo de execução, o código verifica a presença da ponte do Capacitor para decidir a origem das posições. No aplicativo nativo, utiliza o plugin de *background geolocation*; no navegador, recorre à *Geolocation API*. Ambos os caminhos alimentam o **mesmo** núcleo de cálculo, garantindo consistência de resultados entre plataformas.

```
// Detecção do ambiente nativo e obtenção do plugin
const isNative = !(window.Capacitor
  && window.Capacitor.isNativePlatform
  && window.Capacitor.isNativePlatform());
const BackgroundGeolocation = (isNative && window.Capacitor.registerP
  ? window.Capacitor.registerPlugin('BackgroundGeolocation')
  : null;
```

4.3 Estrutura do repositório

Tabela 3 — Organização dos principais arquivos.

Arquivo / diretório	Função
<code>index.html</code>	Aplicação completa (interface + lógica)
<code>geomag.bundle.js</code>	Modelo WMM2025 empacotado (declinação offline)
<code>manifest.json</code> , <code>icon-*.png</code>	Metadados e ícones do PWA
<code>capacitor.config.json</code>	Identificador e configuração do app nativo
<code>scripts/copy-web.mjs</code>	Gera a pasta web e injeta versão/commit no build
<code>android/</code>	Projeto Android (permissões de background no manifesto)
<code>.github/workflows/android.yml</code>	Compilação e publicação do APK
<code>CHANGELOG.md</code> , <code>NATIVE.md</code>	Histórico de versões e guia de build

4.4 Fluxo de dados do rastreamento

Ao iniciar, o aplicativo registra um observador de posição (nativo ou web). Cada leitura recebida é encaminhada a duas rotinas: a de *renderização*, que atualiza os campos de posição, precisão, velocidade e a bússola; e a de *ingestão*, que decide se, e quanto, adicionar à distância acumulada. O estado da sessão — distância, tempo, último ponto — é persistido de forma contínua, permitindo retomada após fechamento inesperado.

5 DESENVOLVIMENTO E DESCRIÇÃO DO CÓDIGO

5.1 Obtenção e refinamento da posição

A leitura pontual de posição sofre de um vício conhecido: o primeiro *fix* costuma ser a estimativa grosseira de rede. Para contorná-lo, a rotina de aquisição não aceita a

primeira resposta; ela abre um observador temporário por até trinta segundos e retém, dentre as leituras, a de **menor raio de erro**, encerrando antecipadamente assim que a precisão atinge o alvo de 20 m. Enquanto refina, informa o usuário do valor corrente e, ao final, classifica a precisão em *boa* (≤ 20 m), *moderada* (≤ 200 m) ou *aproximada* (> 200 m), orientando, neste último caso, a ativação da localização precisa no aparelho.

5.2 Diagnóstico de localização aproximada

Um sintoma recorrente relatado em campo — "a posição acerta apenas a cidade" — decorre da concessão, pelo sistema, de *localização aproximada* em vez de *precisa*. O aplicativo detecta esse regime pela precisão em nível de centenas ou milhares de metros e emite orientação explícita, em vez de silenciosamente registrar um ponto errado.

5.3 Evolução do cálculo de distância

O cálculo de distância foi o componente mais refinado do projeto, precisamente porque é onde o ruído do GPS se manifesta de forma mais nociva. A primeira versão somava toda variação de posição superior a meio metro; o resultado, em campo, era uma distância fortemente inflada, pois o tremor do sinal excede facilmente esse limiar. As iterações seguintes elevaram o limiar e o tornaram proporcional à precisão, reduzindo — mas não eliminando — a superestimação. A introdução do filtro por velocidade resolveu o caso do aparelho imóvel. A síntese desses aprendizados é o método híbrido descrito a seguir.

5.4 O método híbrido de distância

O método híbrido combina três mecanismos. O **primeiro** é a integração da velocidade: quando o GPS reporta velocidade confiável, a distância entre duas amostras é a média das velocidades multiplicada pelo intervalo de tempo, com *auto-pausa* quando a velocidade corrente está abaixo do limiar de repouso. O **segundo** é a rejeição de *spike*: um ponto cuja velocidade implícita pela posição seja fisicamente impossível, ou muito superior à velocidade Doppler reportada, é descartado sem mover a referência, impedindo saltos espúrios. O **terceiro** é o fallback por posição, acionado quando a velocidade não está disponível, empregando o limiar proporcional à precisão para conter o zigue-zague. A Tabela 4 lista os parâmetros de calibração.

Tabela 4 — Constantes de calibração do método de distância.

Constante	Valor	Significado
ACCEPT_ACCURACY_M	25 m	Precisão máxima aceita; acima disso a leitura é descartada
MIN_MOVE_M	15 m	Piso do limiar de movimento (fallback por posição)
NOISE_FACTOR	1,5	Limiar = $\text{máx}(\text{MIN_MOVE_M}, 1,5 \times \text{precisão})$
MIN_SPEED_MS	0,5 m/s	Limiar de auto-pausa ($\approx 1,8$ km/h)
MAX_SPEED_MS	55 m/s	Velocidade impossível (≈ 200 km/h) $\rightarrow$ <i>spike</i>
SPIKE_FACTOR	3	Fator sobre a velocidade Doppler para detectar <i>spike</i>
MAX_DT_S	30 s	Intervalo máximo para integrar velocidade
NATIVE_DISTANCE_FILTER_M	10 m	Espaçamento mínimo entre leituras do plugin nativo

O trecho a seguir sintetiza o núcleo de ingestão de posição:

```

// Núcleo híbrido (resumido)
if (acc > ACCEPT_ACCURACY_M) return; // 0) descarta ruído
const dt = (now - lastTime) / 1000;
const dPos = haversine(lastPos, pos);
const implied = dt > 0 ? dPos / dt : 0;

// #2 rejeição de spike / teleporte
if (implied > MAX_SPEED_MS ||
    (spRef !== null && implied > spRef * SPIKE_FACTOR + 5)) return;

let added = 0;
if (spRef !== null && dt > 0 && dt <= MAX_DT_S) {
  // #1 integração da velocidade + auto-pausa pela velocidade atual
  if (spRef >= MIN_SPEED_MS) {
    const prevMoving = (lastSpeed !== null && lastSpeed >= MIN_SPEED_MS);
    const v = prevMoving ? (lastSpeed + spRef) / 2 : spRef;
    added = v * dt;
  }
} else {
  // fallback por posição, com limiar proporcional à precisão
  const threshold = Math.max(MIN_MOVE_M, NOISE_FACTOR * Math.max(acc,
    if (dPos >= threshold) added = dPos; else { /* mantém referência */
  }
}
totalDistance += added;

```

5.5 Continuidade: Wake Lock, costura e persistência

Para o modo web, três mecanismos mitigam a suspensão. O **Wake Lock** mantém a tela acesa durante o rastreamento, reduzindo a probabilidade de suspensão. A **costura** (*bridging*) reconcilia o trecho perdido: ao retornar do segundo plano, o aplicativo obtém um novo *fix* e soma, em linha reta, o deslocamento desde o último ponto salvo. A **persistência** grava continuamente distância, tempo e último ponto no armazenamento local, permitindo que uma sessão interrompida seja *retomada* — inclusive após o encerramento completo do aplicativo — pelo botão de retomada, que dá prosseguimento à contagem sem reiniciá-la.

5.6 Rastreamento nativo em segundo plano

No aplicativo Android, o núcleo é alimentado pelo plugin de *background geolocation*, que mantém um serviço em primeiro plano com notificação persistente. Como o próprio sistema entrega as posições mesmo com a tela bloqueada, os mecanismos de costura tornam-se dispensáveis nesse modo. O projeto nativo declara, em seu manifesto, as permissões de localização em primeiro e segundo plano necessárias, e o guia de compilação documenta o processo de geração do APK.

5.7 Bússola de norte verdadeiro

A bússola combina o rumo do magnetômetro com a declinação local. No Android, o rumo provém do evento de orientação *absoluto*, referenciado ao norte; no iOS, do rumo de bússola específico do sistema. A declinação é obtida do modelo WMM2025, avaliado para a posição corrente, e a relação empregada — a mesma que o calculador oficial do NOAA adota — é:

```
rumo_verdadeiro = rumo_magnético + declinação
```

A rosa dos ventos é então girada de modo que o norte aponte para o norte geográfico na tela. Um modo de diagnóstico exhibe os valores brutos do sensor (ângulo, rumo magnético, ângulo da tela, declinação e rumo verdadeiro), permitindo verificar objetivamente o comportamento do magnetômetro em cada aparelho.

5.8 Interface e experiência do usuário

A interface, de tema escuro e responsiva, organiza-se em cartões: posição atual, bússola, distância percorrida em destaque, e um painel de tempo e velocidades. Os controles principais — obter posição, iniciar, encerrar e **zerar** o rastreamento — são explícitos; o botão de zerar solicita confirmação quando há distância acumulada, evitando perdas acidentais, e opera tanto com o rastreamento ativo (recomeçando do zero sem interrompê-lo) quanto parado. O rodapé exhibe a versão do software e, no aplicativo nativo, o identificador exato da compilação.

6 VALIDAÇÃO E RESULTADOS

6.1 Verificação da fórmula de distância

A implementação da Haversine foi conferida contra distâncias de referência conhecidas, confirmando sua correção em escalas de metros a milhares de quilômetros (Tabela 5).

Tabela 5 — Verificação da fórmula de Haversine.

Par de pontos	Calculado	Referência
0,001° de latitude	111,19 m	111,19 m
São Paulo → Rio de Janeiro	360,7 km	≈ 361 km
Equador → Polo	10.008 km	10.008 km

6.2 Simulação comparativa do cálculo de distância

A Tabela 6 sintetiza a comparação entre os métodos, com ruído correlacionado, para trajetos de distância verdadeira conhecida. Evidencia-se a superestimação severa do método por posição pura, em especial sob baixa precisão, e a acurácia do método híbrido em todos os cenários — inclusive o de descontinuidade, em que um único ponto de teleporte de 500 m infla o método por posição para além de 1.100 m, ao passo que o híbrido o descarta.

Tabela 6 — Distância estimada (m) por método frente ao valor verdadeiro.

Cenário	Verdadeiro	Posição	Híbrido
Reto 100 m, precisão ~8 m	100	100	99
Reto 100 m, precisão ~24 m	100	178	102
Trajeto sinuoso	211	219-279	199-202
Imóvel por 3 min	0	0-424	0
Reto 100 m + teleporte 500 m	≈100	1121	100
Reto 1000 m, precisão ~16 m	1000	1473	1025

6.3 Validação da declinação magnética

Os valores de declinação computados pelo modelo embarcado foram comparados a referências do NOAA, apresentando concordância dentro de aproximadamente meio grau (Tabela 7) — margem inferior à própria incerteza de um magnetômetro de celular. Em campo, na posição real do usuário, o aplicativo indicou $-22,2^\circ$, coincidindo exatamente com o calculador oficial.

Tabela 7 — Declinação calculada versus referência NOAA (2026).

Localidade	Calculado	Referência
São Paulo	$-21,9^\circ$	$\approx -22^\circ$
Tóquio	$-7,9^\circ$	$\approx -7,9^\circ$
Colorado (EUA)	$+7,6^\circ$	$\approx +7,4^\circ$

6.4 Testes no código do aplicativo

Além das simulações isoladas, o próprio código do aplicativo foi exercitado com um observador de posição substituto, alimentado por sequências realistas contendo velocidade e marcas de tempo. Uma caminhada de 100 m resultou em 100,3 m; um ponto de teleporte de 500 m foi ignorado; um repouso de dois minutos não acumulou distância; e a retomada da caminhada por mais 50 m elevou o total a 150,6 m, confirmando o comportamento esperado do método e da função de zerar.

6.5 Compilação e distribuição

O fluxo de integração contínua produziu, de forma reprodutível, o APK de depuração a cada alteração, publicando-o para download direto. O rodapé de cada compilação exibe versão e identificador de *commit*, tornando trivial associar um APK instalado à linha exata do histórico — recurso solicitado explicitamente para evitar confusão entre builds.

SÍNTESE DOS RESULTADOS

O método híbrido reduziu a superestimação de distância de fatores de 2 a $11\times$ (método por posição sob ruído) para erros da ordem de poucos por cento, eliminou a

distância fantasma em repouso e neutralizou descontinuidades do sinal; a bússola de norte verdadeiro reproduziu a referência oficial dentro de meio grau.

7 LIMITAÇÕES

O método híbrido depende da disponibilidade da velocidade Doppler; em dispositivos ou navegadores que não a reportam, o sistema recai no método por posição, de acurácia inferior. A costura de trecho em segundo plano, no modo web, aproxima por linha reta o caminho efetivamente percorrido, subestimando trajetos curvos realizados com o aplicativo suspenso — razão pela qual o modo nativo é preferível para rastreamento contínuo. A bússola está sujeita à necessidade de calibração do magnetômetro e a variações de convenção de eixos entre fabricantes de aparelhos Android. Por fim, a fórmula de Haversine assume um modelo esférico da Terra; embora irrelevante nas distâncias entre amostras, essa aproximação existe.

8 TRABALHOS FUTUROS

- **Exportação de trajeto** em formatos GPX/KML, para análise em ferramentas de mapa;
- **Mapa ao vivo** com o traçado desenhado, mantendo o princípio de autocontenção;
- **Filtro de Kalman** leve sobre a trajetória, para reduzir ainda mais a subestimação em curvas;
- **Histórico de sessões** persistido no aparelho, com estatísticas agregadas;
- **Empacotamento para iOS**, reaproveitando o mesmo código sob o Capacitor;
- **Assinatura de release** do APK, para distribuição fora do modo de depuração.

9 CONCLUSÃO

Este relatório documentou a concepção, a implementação e a validação do GPS Tracker, um aplicativo que, partindo de um requisito mínimo, amadureceu até um sistema multiplataforma com tratamento rigoroso do sinal de posicionamento. A principal contribuição técnica é o **método híbrido de distância**, que, ao privilegiar a velocidade Doppler e rejeitar discontinuidades, supera a estratégia de soma de posições adotada por ferramentas populares e conhecida por superestimar. As decisões de projeto foram sistematicamente ancoradas em **validação quantitativa** — por simulação com ruído realista e por testes de execução — e não em impressões subjetivas, prática que se mostrou decisiva para diagnosticar e corrigir a inflação de distância relatada em campo. O resultado é um aplicativo preciso, instalável, capaz de rastrear em segundo plano e de indicar o norte geográfico, mantido em um único documento auditável e distribuído por um pipeline automatizado. As limitações identificadas e os trabalhos futuros propostos delineiam um caminho natural de continuidade, sem comprometer os princípios de simplicidade e portabilidade que orientaram o desenvolvimento.

REFERÊNCIAS

CAPACITOR. *Capacitor Documentation*: cross-platform native runtime for web apps. Ionic, 2026. Disponível em: <https://capacitorjs.com/docs>. Acesso em: 15 jul. 2026.

MDN WEB DOCS. *Geolocation API*. Mozilla, 2026. Disponível em: https://developer.mozilla.org/docs/Web/API/Geolocation_API. Acesso em: 15 jul. 2026.

MDN WEB DOCS. *Screen Wake Lock API*. Mozilla, 2026. Disponível em: https://developer.mozilla.org/docs/Web/API/Screen_Wake_Lock_API. Acesso em: 15 jul. 2026.

NATIONAL CENTERS FOR ENVIRONMENTAL INFORMATION. *World Magnetic Model (WMM2025)*. NOAA/NCEI, 2025. Disponível em: <https://www.ncei.noaa.gov/products/world-magnetic-model>. Acesso em: 15 jul. 2026.

NATIONAL CENTERS FOR ENVIRONMENTAL INFORMATION. *Magnetic Field Calculators*. NOAA/NCEI, 2026. Disponível em: <https://www.ngdc.noaa.gov/geomag/calculators/magcalc.shtml>. Acesso em: 15 jul. 2026.

SINNOTT, R. W. Virtues of the Haversine. *Sky and Telescope*, v. 68, n. 2, p. 159, 1984.

STRAVA. *How Distance is Calculated*. Strava Support, 2026. Disponível em: <https://support.strava.com/hc/en-us/articles/216919487>. Acesso em: 15 jul. 2026.

NATURAL ATLAS. *geomagnetism*: a JavaScript implementation of the World Magnetic Model. GitHub, 2025. Licença Apache-2.0. Disponível em: <https://github.com/naturalatlas/geomagnetism>. Acesso em: 15 jul. 2026.

ANDROID DEVELOPERS. *Foreground services and background location*. Google, 2026. Disponível em: <https://developer.android.com/develop/sensors-and-location>. Acesso em: 15 jul. 2026.